A Benchmark Generator of Realistic Logistic Transport Networks with Nonlinear Edge Costs and Transshipment Capacities

Varvara Kovaleva[1] and Alexander Ponomarenko[2]

[1] HSE University, Nizhny Novgorod, Russia
varvara_kovaleva03@mail.ru
[2] Laboratory of Algorithms and Technologies for Network Analysis, HSE University, Nizhny Novgorod, Russia
aponomarenko@hse.ru

Abstract. Routing many commodities through a logistic network at minimum joint cost of vehicle movements and transshipment at intermediate storages becomes substantially harder—and substantially more realistic—once two features are present together: edge costs that are *nonlinear* in the transported volume, because freight travels in fixed-capacity vehicles, and *hard upper bounds* on the volume reloaded at each storage. No public benchmark captures this combination, and the locations of commercial distribution centres are proprietary. We close this gap with a parameterised generator of synthetic logistic networks, calibrated on real road graphs of nine European countries, five U.S. states and the European part of Russia extracted from OpenStreetMap. The generator reproduces the structural signatures of upper-level distribution networks—near-planar topology, low vertex degree, short edges and large diameters—by combining Zipf-distributed city sizes, a spatial-network budget model trading edge-building cost against routing convenience, centrality-driven storage capacities, and a doubly constrained gravity demand model. We release 35 fully synthetic instances (10–150 storages) and 15 real-geometry instances in an open, self-describing, fully reproducible file format. An empirical characterisation computed directly from the released files shows that synthetic and real-geometry instances are structurally consistent (two-sample Kolmogorov–Smirnov distances of 0.10 on degree and edge-length distributions, with small and quantified mean deviations), and a baseline-solver study demonstrates that the corpus discriminates instance difficulty across orders of magnitude. The benchmark is a faithful, reusable testbed for the development and comparison of exact and heuristic algorithms.

Keywords: Benchmark generation · Synthetic data · Spatial networks · Multi-commodity flow · Gravity model · Zipf's law · Reproducibility · Logistics · Transportation · Distribution · Transshipment · Operations research · Combinatorial optimisation · Mixed integer linear programming

1 Introduction

Planning commodity flows between distribution centres is a core problem for logistic operators, and its realistic variants are computationally demanding. Two features make real instances especially difficult. First, the cost of using a road is *nonlinear* in the volume carried: goods move in vehicles of fixed capacity, so the billed quantity is the integer number of vehicles obtained by rounding the carried volume up to a vehicle capacity. Second, every storage has a finite *transshipment capacity*, an upper bound on the volume that can be unloaded, stored and reloaded there. The resulting model is a capacitated multi-commodity flow problem with a step-wise vehicle cost—it is NP-hard, and its difficulty is sensitive to both the topology and the demand pattern of the underlying network.

Progress on solution algorithms—both exact methods and heuristics—for such problems is hampered by the absence of suitable public benchmarks. Classical multi-commodity flow generators and libraries do not carry node-level transshipment capacities, per-unit reload costs or vehicle-induced nonlinear edge costs, and they rarely reflect the geographic structure of country-scale distribution networks. Moreover, the true locations of commercial distribution centres are proprietary. Researchers are therefore left to test on either toy graphs or ad-hoc random graphs whose structure differs sharply from real transport networks, which undermines the external validity of any comparison. A generator that produces *plausible* instances at controllable scale, together with a released, citable corpus, is needed to make comparisons of exact and heuristic algorithms meaningful and reproducible.

Contributions. This paper introduces such a generator and corpus. Specifically, we

- define precisely the data an instance of the problem must contain (Section 3);
- analyse real road graphs from OpenStreetMap and distil, with per-region measurements, the structural properties a realistic generator must reproduce (Section 4);
- present a five-stage generation pipeline, with explicit algorithms, combining Zipf city sizes, a spatial-network budget model, centrality-driven storage capacities and a gravity demand model (Section 5);
- describe the released benchmark, its open file format and reproducibility guarantees (Section 6);
- characterise the corpus empirically and directly from the released files, including two-sample Kolmogorov–Smirnov tests against the real networks (Section 7); and
- demonstrate, through a baseline-solver study, that the corpus discriminates instance difficulty across scales (Section 8).

2 Related Work

Flow-problem instance generators. Synthetic generators for network-flow problems have a long history. NETGEN [1] produces capacitated transportation and

min-cost flow instances by random assignment of arcs, costs and capacities, and the DIMACS implementation challenges [2] standardised families of max-flow and min-cost-flow instances. These generators target the combinatorial structure of the flow program but are agnostic to geography: arcs and costs are sampled without spatial embedding, so the resulting graphs do not exhibit the planar, short-edge character of road networks, and they do not model node-level transshipment capacities or vehicle-induced step costs. Our generator instead starts from a realistic spatial topology and layers the logistic cost structure on top.

Random and spatial network models. Purely combinatorial models—Erdős–Rényi random graphs [3], Watts–Strogatz small-world graphs [4] and Barabási–Albert scale-free graphs [5]—reproduce specific statistics (degree, clustering, path length) but not the geometry of transport networks. Spatial network models are closer to our needs: the Waxman model [6] connects nodes with a distance-decaying probability, and Gastner and Newman [7] analyse real spatial networks (road, internet and airline) and propose a construction that trades total edge length against routing convenience. Barthélemy's survey [8] documents the near-planarity, low degree and large diameter of road networks. We adopt the Gastner–Newman trade-off as the topological core of our generator and calibrate its parameters against real road graphs.

City sizes and travel demand. The rank–size distribution of city populations follows Zipf's law [10], which we use to seed realistic node "masses". Travel and freight demand between regions is classically modelled by the gravity analogy; the doubly constrained gravity model of Wilson [11], a staple of transport planning [12], produces origin–destination matrices consistent with prescribed marginals. We combine these ingredients—rarely assembled together in a single flow-problem generator—to obtain instances that are realistic in topology, capacity and demand simultaneously. Finally, we follow FAIR data principles [15] in releasing a versioned, citable corpus.

Heuristics for related logistic optimisation problems. Closely related computational work has addressed vehicle-level logistic routing rather than upper-level flow planning. Batsyn and Ponomarenko proposed a multi-start greedy heuristic for a real-life truck and trailer routing problem arising in retail distribution [16], and extended the approach to a site-dependent variant with soft and hard time windows and split deliveries [17]. Such methods route vehicles once the demand at each storage is fixed; the problem targeted by the present benchmark—multi-commodity transport between storages with nonlinear edge costs and transshipment capacities—sits one level higher in the planning hierarchy and determines those demands. The two problem classes are complementary, and we expect the corpus released here to support analogous heuristic development at the network-flow level.

3 The Optimisation Problem an Instance Must Support

To fix the data requirements we briefly state the target problem. The network is a directed graph $G = (V, E)$ of storages V and roads E. A set of transportation requests (commodities) K is given; each $k \in K$ has an origin s_k, a destination t_k and a demand $d_k \in \mathbb{R}_+$. Each storage v has a per-unit reload cost $f_v \in \mathbb{R}_+$ and a maximum transshipment volume $W_v \in \mathbb{R}_+$; each edge e has a per-vehicle traversal cost $c_e \in \mathbb{R}_+$; and vehicles have a fixed capacity C. Writing x_e^k for the flow of commodity k on edge e and $y_e \in \mathbb{Z}_+$ for the number of vehicles on e, the problem minimises movement plus reload cost,

$$\min \sum_{e \in E} c_e \, y_e \; + \sum_{k \in K} \sum_{v \neq s_k,\, v \neq t_k} \sum_{e \in \delta^-(v)} f_v \, x_e^k, \tag{1}$$

subject to flow conservation for every commodity, the vehicle-capacity coupling $\sum_k x_e^k \leq C \, y_e$ for all $e \in E$, and the storage-capacity bound $\sum_k \sum_{e \in \delta^-(v)} x_e^k \leq W_v$ for all transit v. Because the y_e are integer and the flows continuous, the model is a mixed integer linear program (MILP), and the vehicle term makes the effective edge cost a step function of load.

Consequently, a complete instance must specify: a weighted graph with vertex coordinates; per-storage reload cost f_v and capacity W_v; per-edge cost c_e (here proportional to road length); a vehicle capacity C; and an origin–destination demand set $\{(s_k, t_k, d_k)\}$. The generator produces exactly these objects, together with auxiliary data (populations, all-pairs shortest distances) used to derive them and useful to solvers.

4 Structural Properties of Real Networks

Since the locations of real distribution centres are not public, we calibrate on city locations, populations and highways from OpenStreetMap [13], for nine European countries of varying size, five large U.S. states, and the European part of Russia. From the raw highway graph and city list we aggregate nearby road endpoints into nodes, select cities greedily by descending population subject to a minimum spacing, attach each retained city to a nearby high-degree road node (junctions are economical places for a centre), and finally simplify the graph so that each retained centre is joined to others by edges equal to the shortest stop-free paths. Edges are then treated as straight segments of Euclidean length, since modelling the true road geometry would over-complicate synthesis without materially changing the network structure.

Table 1 reports the resulting per-region measurements. Three structural signatures emerge robustly and become the design targets of the generator. First, hop diameters are large relative to size (up to 16 for 90 nodes); together with a right-skewed, short-edge length distribution this indicates the *opposite* of a small-world structure—routes between distant nodes traverse many short edges. Second, the mean vertex degree is low (about 3.6 across the calibration graphs), well below the planar upper bound of 6, so the networks are essentially planar.

Table 1. Structural measurements of the 15 calibration networks and the spatial-network parameters fitted to each. *deg*: mean vertex degree; *diam*: hop diameter; α: fitted Zipf exponent of city sizes; λ, $coeff_{\text{budget}}$: spatial-network model parameters (Section 5.3).

Network	$\lvert V\rvert$	*deg*	*diam*	α	λ	Network	$\lvert V\rvert$	*deg*	*diam*	α	λ
Denmark	11	2.55	5	0.91	0.23	Sweden	45	3.42	11	1.04	0.23
Belgium	16	3.13	5	0.76	0.25	Italy	46	3.61	11	0.76	0.33
Netherlands	18	3.67	5	0.89	0.03	Germany	49	5.39	8	0.97	0.35
Hungary	19	2.74	7	1.01	0.23	Texas	58	3.00	10	1.20	0.28
Ohio	22	3.73	6	0.87	0.03	Spain	59	4.14	9	0.96	0.50
Georgia	24	3.75	7	0.89	0.08	Eur. Russia	90	4.27	16	1.34	0.50
Illinois	28	3.50	6	0.88	0.30	*mean*	–	3.57	–	0.97	0.26
California	39	2.56	12	1.01	0.03	UK	42	4.19	9	1.13	0.63

Third, the rank–size relation of city populations is approximately log-linear, with a Zipf exponent α averaging 0.97—consistent with the empirical range reported for city systems [10]. The columns λ and $coeff_{\text{budget}}$ are the spatial-network parameters (Section 5.3) that best reproduce each region's diameter, mean edge length and mean degree.

5 The Generation Pipeline

Given a desired number of storages k, the generator proceeds in five stages: city sizes, spatial layout, topology, capacities and demand. Each stage is parameterised by distributions fitted to the calibration data, so that drawing parameters per instance yields a diverse but realistic family.

5.1 Stage 1 – City sizes via Zipf's law

A pool of cities (several times larger than k) is created with sizes following Zipf's law [10], $size_i = P/\,rank_i^{\alpha}$, where P is the largest city's size and small multiplicative noise is added for realism. From the calibration regions (Table 1) the exponent clusters near $\alpha \approx 0.97$, so the generator draws $\alpha \sim \mathcal{N}(0.974, 0.06)$, the largest population from $(9, 11) \cdot 10^4\, k$, and a city count of k times a random integer in $[8, 12]$.

5.2 Stage 2 – Spatial layout and storage selection

The k city clusters are placed on a square map with a Gaussian blob model (cluster standard deviation about 1/15 of the map side), seeding the largest city in each cluster; this imitates the real pattern of smaller towns surrounding a large city. Storages are then selected greedily and assigned the summed population of their nearest cities (Algorithm 1). The minimum spacing between storages is a random value in $[80, 100]$ and the map side scales as that spacing times $k^{0.8}$.

input: cities $\{(p_i, pop_i)\}$, target count k, min. spacing ρ
sort cities by descending pop_i; $S \leftarrow \varnothing$
for each city i in sorted order **do**
  if $\min_{u \in S} \|p_i - p_u\| \geq \rho$ **then** $S \leftarrow S \cup \{i\}$
  if $|S| = k$ **then break**
assign each city i to its nearest storage $u(i) = \arg\min_{u \in S} \|p_i - p_u\|$
for each storage $u \in S$: $W_u^{\mathrm{pop}} \leftarrow \sum_{i:\, u(i)=u} pop_i$
return storages S with aggregated populations W^{pop}

Algorithm 1: Greedy storage selection and population assignment.

input: storage positions, λ, $coeff_{\mathrm{budget}}$
$T_0 \leftarrow \mathrm{MST}(V)$; $B \leftarrow coeff_{\mathrm{budget}} \cdot \mathrm{cost}(T_0)$
$G \leftarrow T_0$; evaluate $\Phi(G)$ = mean pairwise effective distance
repeat (annealing schedule)
  propose add/remove of a candidate edge keeping $\mathrm{cost}(G) \leq B$
  accept with Metropolis probability w.r.t. $\Delta\Phi$
until converged
return connected graph G within budget

Algorithm 2: Budget-constrained topology by simulated annealing.

5.3 Stage 3 – Topology via a spatial-network budget model

Following Gastner and Newman [7], edges are chosen to balance construction cost against routing convenience. Construction cost is the total edge length $\sum_{(i,j)} d_{ij}$ (Euclidean, in km), while the routing "effective distance" of an edge is

$$L_{ij} = \lambda\, d_{ij} + (1 - \lambda) \cdot 1, \qquad 0 \leq \lambda \leq 1, \tag{2}$$

so that $\lambda = 1$ measures routes in kilometres and $\lambda = 0$ in hop count. Starting from a minimum spanning tree (which guarantees connectivity and sets the budget reference), edges are added within a budget—a coefficient $coeff_{\mathrm{budget}}$ times the MST cost—so as to minimise mean pairwise effective distance. We solve this combinatorial trade-off by simulated annealing (Algorithm 2). Matching the diameter, mean edge length and mean degree of the calibration graphs yields $\lambda \sim \mathcal{N}(0.263, 0.15)$ and $coeff_{\mathrm{budget}} \sim \mathcal{N}(2.56, 0.6)$; a small budget admits only a few long edges among many short ones, exactly the observed real pattern. Each edge is duplicated in reverse with length differing by at most one percent, and per-vehicle cost is $c_e = cost_{km} \cdot dist_e$ with $cost_{km} = 5$.

5.4 Stage 4 – Storage capacities and reload costs

A storage's transshipment capacity should reflect both the demand it serves (population) and the through-flow it attracts. The latter correlates strongly with betweenness centrality [8,9], so the capacity is

$$W_v = l \cdot population_v^{\gamma}\, (1 + \alpha_b\, \tilde{b}_v), \tag{3}$$

input: marginals O, D; deterrence $f(dist_{ij})$; tolerance ε
$A_i \leftarrow 1,\ B_j \leftarrow 1$ for all i, j
repeat
  $A_i \leftarrow \left(\sum_j B_j D_j f(dist_{ij})\right)^{-1}$ for all i
  $B_j \leftarrow \left(\sum_i A_i O_i f(dist_{ij})\right)^{-1}$ for all j
until max relative change of $A, B < \varepsilon$
return $T_{ij} = A_i O_i B_j D_j f(dist_{ij})$

Algorithm 3: Doubly constrained gravity balancing.

where $\tilde{b}_v$ is normalised betweenness, with $l = 1, \gamma = 0.5, \alpha_b = 1.2$ and a floor $W_{\min} = 100$ (capacities are scaled so at least 10% fall below the floor). Reload costs decrease with capacity, reflecting economies of scale: $f_v = \kappa / W_v^{\delta}$ with $\kappa = 500, \delta = 1$.

5.5 Stage 5 – Demand via a doubly constrained gravity model

The origin–destination matrix uses a doubly constrained gravity model [11,12],

$$T_{ij} = A_i\, O_i\, B_j\, D_j\, f(dist_{ij}), \qquad f(dist) = dist^{-\beta},\ \beta = 1, \tag{4}$$

where the marginals O_i, D_j are proportional to served population (plus small noise), $dist_{ij}$ is the shortest network distance, and the balancing factors A_i, B_j enforce $\sum_j T_{ij} = O_i$ and $\sum_i T_{ij} = D_j$, computed by the standard iterative fitting of Algorithm 3. Total flow is scaled to a few times the total storage capacity so that the capacity constraints bind. Finally 25% of requests are dropped with probability proportional to their volume, so that not every pair exchanges goods, as in real demand matrices.

6 The Benchmark Corpus

The corpus has two parts. The **synthetic** set, produced fully by the pipeline, contains 35 instances: 10 with 10 storages, 7 each with 20 and 30, 5 with 50, 3 with 100 and 3 with 150. The **real-geometry** set contains 15 instances built on the calibration regions (11 to 90 storages): the vertex coordinates and populations come from OpenStreetMap, while edge costs, storage capacities, reload costs and demand are generated by Stages 3–5. Together this spans 10 to 150 storages and 67 to over 16,000 commodities.

Each instance is a directory of plain, self-describing files (Table 2): an undirected weighted graph in GraphML with vertex coordinates, a per-storage table of reload cost and capacity, a city position/population table, the demand requests, a precomputed shortest-distance matrix, and a parameter file recording the random seed and every generation parameter, so that each instance regenerates exactly. The corpus and generator are released under permissive licences (data under CC BY 4.0, code under MIT) following FAIR principles [15]: the

Table 2. File format of a benchmark instance.

File	Contents
`graph.graphml`	weighted graph; node `x,y`; edge `weight` (road length)
`offices.csv`	per storage: `transfer_price` (f_v), `transfer_max` (W_v)
`pos_pop.csv`	city/storage `x, y, population`
`reqs.csv`	demand requests `src, dst, volume` (s_k, t_k, d_k)
`distance_matrix.csv`	all-pairs shortest road distances
`data.txt`	random seed and all generation parameters

Table 3. Synthetic instances aggregated by size (n instances each): mean degree, hop diameter, mean edge length, mean storage capacity $\bar{W}$, mean reload cost $\bar{f}$, commodity count $|K|$, mean demand $\bar{d}$ and fitted Zipf exponent α. All values computed from the released files.

$\|V\|$	n	*deg*	*diam*	*dist*	$\bar{W}$	$\bar{f}$	$\|K\|$	$\bar{d}$	α
10	10	3.68	3.3	189.5	266.0	2.63	67	153.4	1.00
20	7	4.06	5.3	261.6	230.4	2.75	285	56.7	0.84
30	7	3.25	7.9	261.9	199.1	3.00	652	36.1	0.79
50	5	3.18	9.4	304.8	182.8	3.22	1837	21.9	0.78
100	3	3.05	12.0	481.0	178.6	3.27	7425	17.2	0.77
150	3	3.15	18.7	477.0	177.4	3.30	16762	12.4	0.75

instance corpus is archived on Zenodo with a versioned DOI (`https://doi.org/10.5281/zenodo.20372456`), and the generator source code is openly available on GitHub (`https://github.com/Varya-K/Optimal-Logistic-Net`); an interactive web viewer (Streamlit/Plotly) ships alongside for inspecting instances and solver outputs.

7 Empirical Characterisation

We characterise the corpus using statistics computed directly from the released files, to verify that the synthetic instances reproduce real network structure. Table 3 summarises the per-size synthetic aggregates; the figures below pool every instance unless stated otherwise.

Topology. Figure 1 shows a synthetic instance beside the Spain network: both display the same near-planar, locally clustered structure with mostly short edges. Figure 2 pools all instances: the degree distribution is concentrated below the planar bound of 6, and the edge-length distribution (normalised by each graph's largest pairwise distance) is right-skewed and dominated by short edges, in both populations. Figure 3 confirms that hop diameter grows faster than the $\sqrt{|V|}$ reference (non-small-world), with synthetic and real points overlapping, and that the storage population rank–size relation is approximately log-linear, as Zipf's law prescribes.

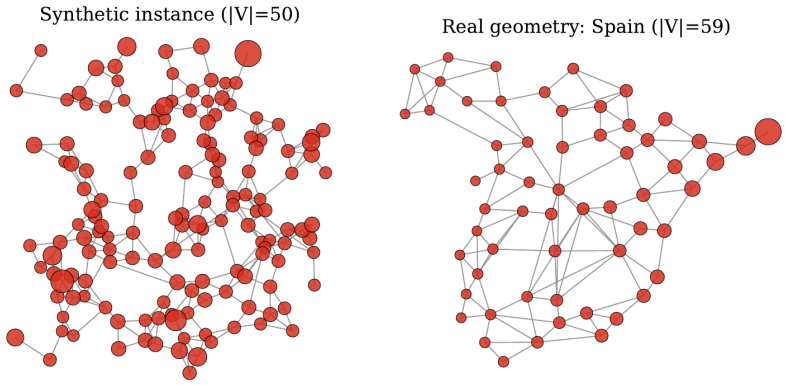


Fig. 1. Generated layouts. Left: a fully synthetic instance ($|V| = 50$). Right: a real-geometry instance (Spain, $|V| = 59$). Node area encodes storage capacity.

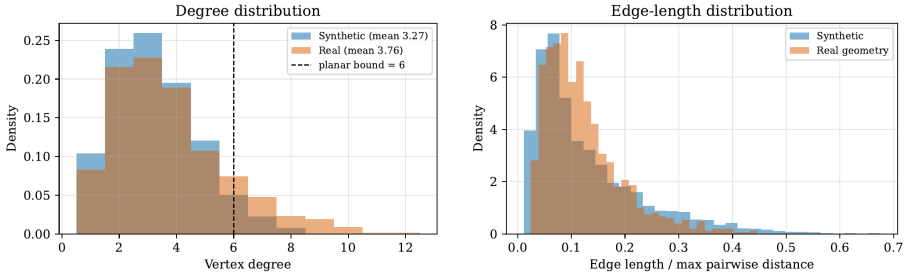


Fig. 2. Pooled topological distributions over all instances. Left: vertex degree (both populations stay below the planar bound 6). Right: normalised edge length (short edges dominate). Synthetic and real-geometry instances are closely aligned.

Quantitative agreement. To test agreement formally we ran two-sample Kolmogorov–Smirnov (KS) tests between the pooled synthetic and real-geometry distributions. The KS distances are small—$D = 0.095$ for vertex degree ($n_{\text{syn}} = 1450$, $n_{\text{real}} = 566$) and $D = 0.103$ for normalised edge length ($n_{\text{syn}} = 2373$, $n_{\text{real}} = 1064$)—confirming that the distributions are close in shape. Both differences are nonetheless statistically significant ($p < 0.01$), which at these large pooled sample sizes reflects modest but real deviations rather than a poor fit: mean degree is 3.27 (95% CI $[3.20, 3.35]$) for synthetic versus 3.76 ($[3.60, 3.92]$) for real, and mean normalised edge length is 0.130 ($[0.126, 0.134]$) versus 0.121 ($[0.117, 0.125]$). In other words, the synthetic networks are slightly sparser and have marginally longer normalised edges, but reproduce the real distributions' location, shape and support faithfully. The same qualitative trends across size (Table 3) hold for the real networks (Table 1), supporting the use of the synthetic set as a controllable proxy for real geography.

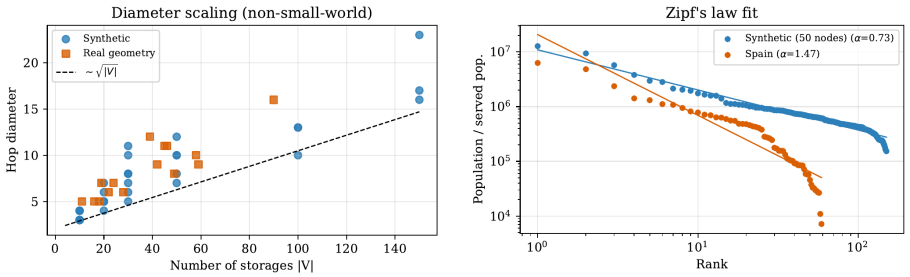


Fig. 3. Left: hop diameter grows faster than $\sqrt{|V|}$ for both populations (networks are not small-world). Right: the rank–size relation of storage populations is approximately log-linear (Zipf).

8 Use Case: Difficulty and Solver Discrimination

A benchmark is only useful if its instances vary in difficulty. We quantify difficulty with three standard, solver-agnostic quantities, all expressed relative to the linear-relaxation optimum (LP), which is a valid lower bound on the true optimum. The first is the LP value itself (ratio 1 by definition). The second is the trivial feasible solution obtained by rounding the relaxation up to integer vehicle counts (Rounded LP); the ratio RoundedLP/LP ≥ 1 brackets the optimality gap and needs no solver beyond a linear program. The third is the best feasible solution returned by a general-purpose exact MILP solver, HiGHS [14], under a wall-clock budget proportional to instance size; the ratio MILP/LP ≥ 1 measures how far a standard exact method gets within a fixed budget. None of these quantities depends on a bespoke algorithm, so the difficulty they reveal is a property of the instances, not of any particular method.

Table 4 and Figure 4 show two clear trends. First, the relaxation gap widens monotonically with size: the trivial upper bound sits 1.23× above the LP lower bound at $|V| = 10$ but 5.6× above it at $|V| = 150$, so the interval that any solver must close grows by roughly an order of magnitude across the corpus. Second, the exact solver degrades and then fails exactly where the gap widens: MILP/LP rises from 1.03 at $|V| = 10$ to 1.94 at $|V| = 50$, and for $|V| \geq 100$ HiGHS returns no feasible solution at all within the budget. The corpus therefore spans the full range from instances an exact solver closes to near-optimality to instances on which it fails outright, which is precisely the discrimination a benchmark must provide. The real-geometry instances (Fig. 4, right) reproduce these trends: the upper-bound ratio ranges from 1.31 on the smallest networks to 3.95 on European Russia ($|V| = 90$), and the exact solver is feasible only up to $|V| = 59$. That the same hardness signature appears on networks built from real geography confirms that difficulty, and not only structure, transfers between the synthetic and real-geometry sets.

Table 4. Instance difficulty on the synthetic set, aggregated by size and expressed as ratios to the LP lower bound (mean over the n instances with reference solutions). RoundedLP/LP brackets the optimality gap; MILP/LP is the time-limited exact solver. "–" marks sizes where HiGHS returned no feasible solution within the budget.

$\|V\|$	n	Rounded LP / LP	Exact MILP / LP
10	10	1.229	1.032
20	7	1.640	1.055
30	7	2.391	1.171
50	5	3.485	1.935
100	3	4.044	–
150	1	5.572	–

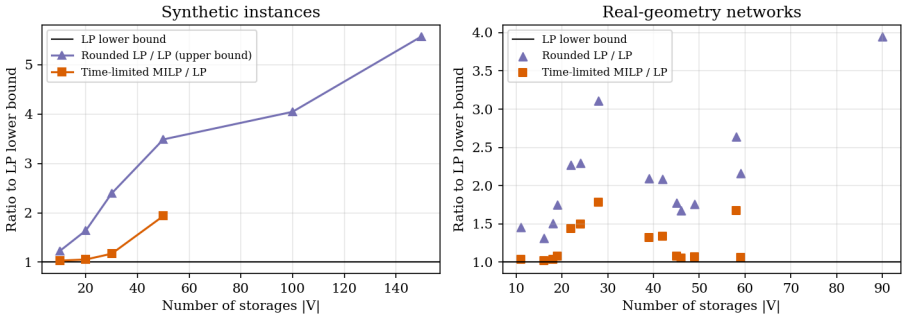


Fig. 4. Instance difficulty as ratios to the LP lower bound versus network size. Left: synthetic instances (means per size). Right: real-geometry networks. The trivial upper bound diverges from the lower bound and the time-limited exact solver becomes infeasible as size grows, demonstrating that the corpus discriminates difficulty without reference to any bespoke solver.

9 Limitations and Future Work

The generator makes deliberate simplifications. Edges are straight segments, so true road sinuosity is not modelled (it changes lengths by a near-constant factor and does not affect topology). Demand is static and symmetricised; time-dependent or stochastic demand is left for future work. Capacities derive from population and betweenness only; richer drivers (warehouse type, modal mix) could be added. The KS analysis shows the synthetic networks are slightly sparser than the real ones, suggesting the budget prior could be widened for larger k. Each of these is a parameter or stage the modular pipeline can extend without redesign, and the released code is structured to make such extensions straightforward.

10 Conclusion

We presented a generator and an open, reproducible corpus for logistic network-flow problems with nonlinear, vehicle-induced edge costs and hard storage transshipment capacities—a problem class for which no realistic public benchmark previously existed. By calibrating on real road graphs and combining Zipf city sizes, a spatial-network budget model, centrality-driven capacities and a gravity demand model, the generator reproduces the near-planar topology, low degree, short edges and large diameters of real upper-level distribution networks. An empirical characterisation computed directly from the released files shows that the 35 synthetic and 15 real-geometry instances are structurally consistent (KS distances ≈ 0.1 with small, quantified mean deviations), and a baseline-solver study demonstrates that the corpus discriminates instance difficulty across orders of magnitude. The benchmark, generator and viewer are released together so that comparisons of exact and heuristic algorithms on this problem can be reproducible and externally valid.

Data availability. The instance corpus is openly available under CC BY 4.0 and archived on Zenodo with a versioned DOI, `https://doi.org/10.5281/zenodo.20372456`. The generator and the interactive viewer are released under the MIT licence in the development repository on GitHub, `https://github.com/Varya-K/Optimal-Logistic-Net`.

A Per-Region Instance Statistics

Table 5 lists the logistic parameters generated for the real-geometry instances: total storage capacity W_{sum}, mean reload cost $\bar{f}$, number of commodities $|K|$ and total demand d_{sum}. All values are computed from the released files and complement the structural measurements of Table 1.

Table 5. Logistic parameters of the 15 real-geometry instances.

Network	W_{sum}	$\bar{f}$	$\lvert K\rvert$	d_{sum}	Network	W_{sum}	$\bar{f}$	$\lvert K\rvert$	d_{sum}
Denmark	3294	2.35	82	8323	Sweden	21373	1.83	1485	85688
Belgium	5703	2.11	180	15717	Italy	14288	2.40	1552	61953
Netherlands	4694	2.73	229	11605	Germany	14276	2.54	1764	43705
Hungary	4566	3.26	256	20566	Texas	27767	2.31	2479	110540
Ohio	7371	2.58	346	22994	Spain	19789	2.38	2566	77666
Georgia	7720	2.57	414	34832	Eur. Russia	42492	2.03	6007	132570
Illinois	10292	2.53	567	24337	UK	24109	1.75	1291	77119
California	36544	1.48	1111	114359					

Acknowledgments. The study was implemented in the framework of the Basic Research Program at HSE University (HSE-BR-2025-080).

References

1. Klingman, D., Napier, A., Stutz, J.: NETGEN: A program for generating large scale capacitated assignment, transportation, and minimum cost flow network problems. Management Science **20**(5), 814–821 (1974)
2. Johnson, D.S., McGeoch, C.C. (eds.): Network Flows and Matching: First DIMACS Implementation Challenge. DIMACS Series, vol. 12. AMS (1993)
3. Erdős, P., Rényi, A.: On random graphs I. Publicationes Mathematicae **6**, 290–297 (1959)
4. Watts, D.J., Strogatz, S.H.: Collective dynamics of 'small-world' networks. Nature **393**, 440–442 (1998)
5. Barabási, A.-L., Albert, R.: Emergence of scaling in random networks. Science **286**(5439), 509–512 (1999)
6. Waxman, B.M.: Routing of multipoint connections. IEEE Journal on Selected Areas in Communications **6**(9), 1617–1622 (1988)
7. Gastner, M.T., Newman, M.E.J.: The spatial structure of networks. The European Physical Journal B **49**(2), 247–252 (2006)
8. Barthélemy, M.: Spatial networks. Physics Reports **499**(1–3), 1–101 (2011)
9. Kirkley, A., Barbosa, H., Barthelemy, M., Ghoshal, G.: From networks to optimal higher-order models of complex systems. Nature Physics **14**, 250–258 (2018)
10. Gabaix, X.: Zipf's law for cities: an explanation. The Quarterly Journal of Economics **114**(3), 739–767 (1999)
11. Wilson, A.G.: A statistical theory of spatial distribution models. Transportation Research **1**(3), 253–269 (1967)
12. Ortúzar, J.de D., Willumsen, L.G.: Modelling Transport, 4th edn. Wiley, Chichester (2011)
13. OpenStreetMap contributors: OpenStreetMap. `https://www.openstreetmap.org` (accessed 2026-04-11)
14. Huangfu, Q., Hall, J.A.J.: Parallelizing the dual revised simplex method. Mathematical Programming Computation **10**(1), 119–142 (2018)
15. Wilkinson, M.D., et al.: The FAIR guiding principles for scientific data management and stewardship. Scientific Data **3**, 160018 (2016)
16. Batsyn, M., Ponomarenko, A.: Heuristic for a real-life truck and trailer routing problem. Procedia Computer Science **31**, 778–792 (2014)
17. Batsyn, M., Ponomarenko, A.: Heuristic for site-dependent truck and trailer routing problem with soft and hard time windows and split deliveries. In: Pardalos, P., Pavone, M., Farinella, G.M., Cutello, V. (eds.) Machine Learning, Optimization, and Big Data (MOD 2015). LNCS, vol. 9432, pp. 65–79. Springer, Cham (2015)